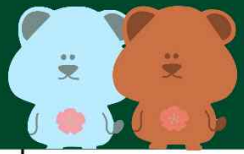


2주차 발표

DL 2팀 김나연 신은서 이시현

EWHA EURON

목차

4장. 딥러닝 시작

#4.1 인공 신경망의 한계와 딥러닝 출현

#4.2 딥러닝 구조

#4.3 딥러닝 및 알고리즘

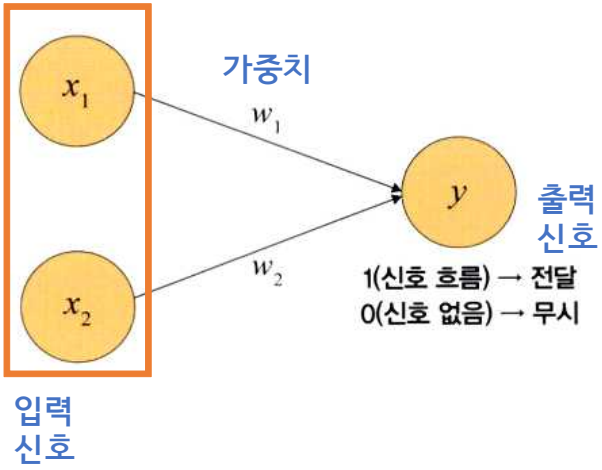
#4.4 우리는 무엇을 배워야 할까?

4.1 인공 신경망의 한계와 딥러닝 출현

#4.1 인공 신경망의 한계와 딥러닝 출현

퍼셉트론

- 오늘날 인공 신경망에서 이용하는 입력층, 출력층, 가중치로 구성된 구조는 Frank Rosenblatt이 고안한 선형 분류기



#4.1 인공 신경망의 한계와 딥러닝 출현

논리 게이트

AND 게이트

모든 입력이 ‘1’일 때 작동

x_1	x_2	y
0	0	0
1	0	0
0	1	0
1	1	1

OR 게이트

입력 1개 이상이
‘1’일 때 작동

x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	1

XOR 게이트

입력 두 개 중 한 개만
‘1’일 때 작동

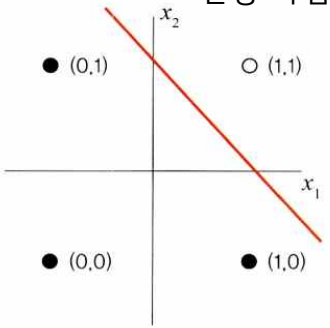
x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	0

#4.1 인공 신경망의 한계와 딥러닝 출현

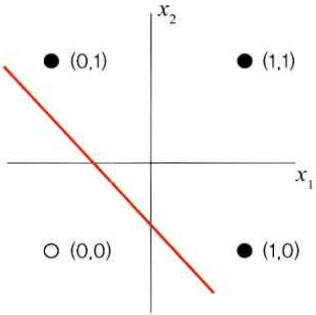
논리 게이트

AND 게이트

- 단층 퍼셉트론에서 선형적 분리 가능
- 단층 퍼셉트론에서 학습 가능

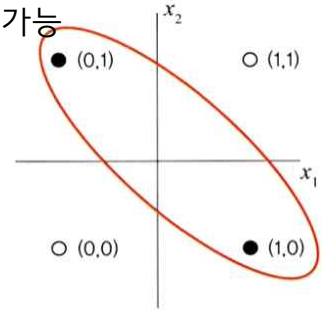


OR 게이트



XOR 게이트

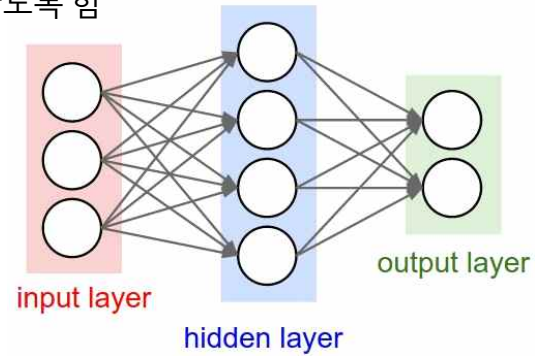
- 데이터가 비선형적으로 분리됨
- 단층 퍼셉트론에서 학습 불가능



#4.1 인공 신경망의 한계와 딥러닝 출현

논리 게이트

- 다층 퍼셉트론
  - 단층 퍼셉트론에서 XOR에 대해서는 학습이 불가능한 문제를 극복하기 위한 방안
  - 입력층과 출력층 사이에 **하나 이상의 중간층(은닉층)**을 두어 비선형적으로 분리되는 데이터에 대해서도 학습이 가능하도록 함

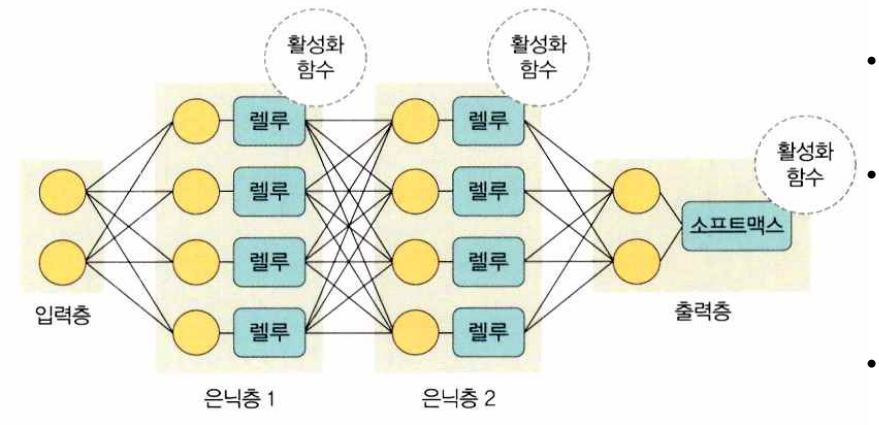


- 심층 신경망 (Deep Neural Network, DNN) = 딥러닝
  : 입력층과 출력층 사이에 은닉층이 여러 개 있는 신경망

4.2 딥러닝 구조

#4.2.1 딥러닝 용어

층



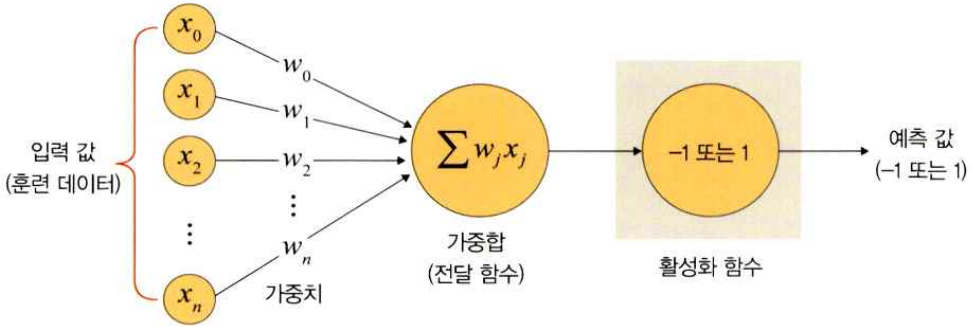
- 입력층 (input layer) : 데이터를 받아들이는 층
- 은닉층 (hidden layer) : 모든 입력 노드부터 입력 값을 받아 가중합을 계산하고, 이 값을 활성화 함수에 적용하여 출력층에 전달하는 층
- 출력층 (output layer) : 신경망의 최종 결과값이 포함된 층

#4.2.1 딥러닝 용어

가중치 (weight)

- 노드와 노드 간 연결 강도
- 입력 값이 연산 결과에 미치는 영향력을 조절하는 요소

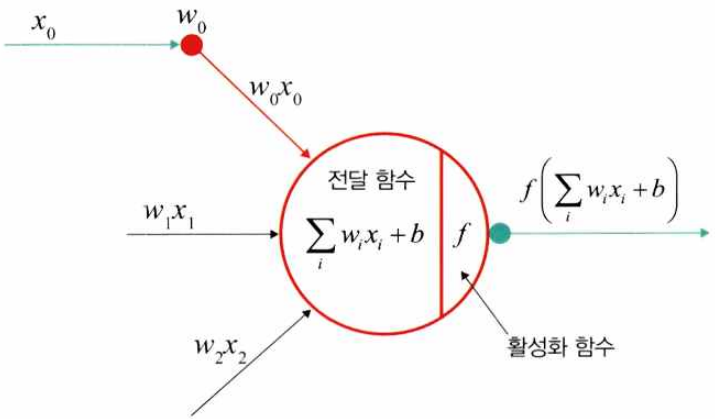
- e.g. 그림
  - w_1 값이 0 혹은 0과 가까운 0.001이라면, x_1 이 아무리 큰 값이어도 $x_1 \times w_1$ 값은 0에 가까운 값이 됨



#4.2.1 딥러닝 용어

가중합 (전달 함수)

- 각 노드에서 들어오는 신호에 가중치를 곱해서 다음 노드로 전달될 때, 이 값들을 모두 더한 합계
- 가중합이 계산되면, 가중합을 활성화 함수로 보내기 때문에 전달 함수(transfer function)라고도 함



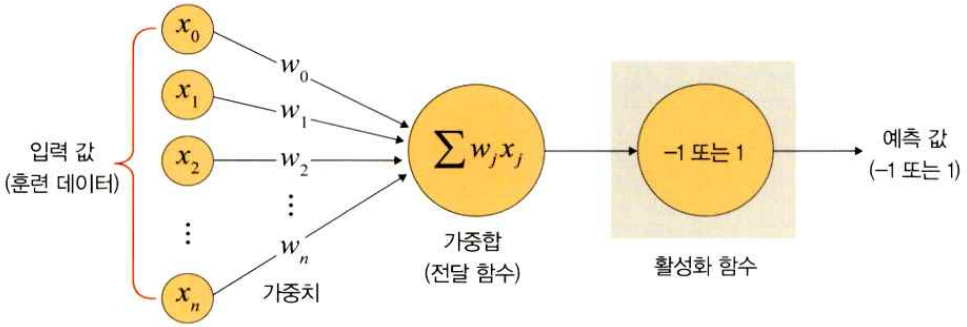
가중합 구하는 공식

$$\sum_i w_i x_i + b$$

(w : 가중치, b : bias)

#4.2.1 딥러닝 용어

가중합 (전달 함수)



가중합 구하는 공식

$$\sum_i w_i x_i + b$$

- Bias : 가중합에 더해주는 상수. 하나의 뉴런에서 활성화 함수를 거쳐 최종적으로 출력되는 값을 조절하는 역할을 함
- bias가 없으면, 입력이 0이면 출력도 무조건 0이 됨

#4.2.1 딥러닝 용어

함수

- **활성화 함수 (Activation function)**
  - 전달 함수에서 신호를 입력받아 이를 적절히 처리하여 출력해 주는 함수
  - 시그모이드 함수, 하이퍼볼릭 탄젠트 함수, ReLU 함수 등이 있음

- **손실 함수 (Loss function)**
  - 가중치 학습을 위해 출력 함수의 결과와 실제 값 간의 오차를 측정하는 함수
  - 값이 클수록 많이 틀렸다는 의미. 값이 ’0’에 가까우면 완벽하게 추정할 수 있다는 의미.
  - 평균 제곱 오차(MSE), 크로스 엔트로피 오차(CEE) 등이 있음

#4.2.1 딥러닝 용어

함수

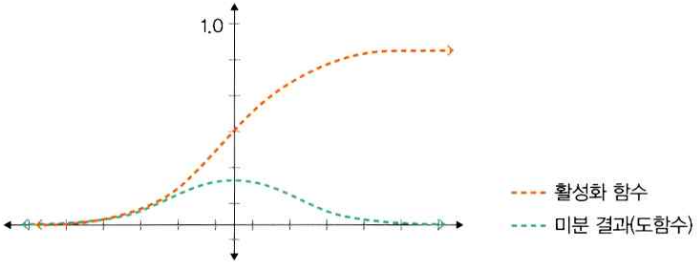
- **활성화 함수 (Activation function)**
  - 전달 함수에서 신호를 입력받아 이를 적절히 처리하여 출력해 주는 함수
  - 시그모이드 함수, 하이퍼볼릭 탄젠트 함수, ReLU 함수 등이 있음

- **손실 함수 (Loss function)**
  - 가중치 학습을 위해 출력 함수의 결과와 실제 값 간의 오차를 측정하는 함수
  - 값이 클수록 많이 틀렸다는 의미. 값이 ’0’에 가까우면 완벽하게 추정할 수 있다는 의미.
  - 평균 제곱 오차(MSE), 크로스 엔트로피 오차(CEE) 등이 있음

#4.2.1 딥러닝 용어

활성화 함수 (Activation function)

- **시그모이드(Sigmoid) 함수** $f(x) = \frac{1}{1+e^{-x}}$
  - 선형 함수의 결과를 0~1 사이에서 비선형 형태로 변형해줌
  - 주로 로지스틱 회귀와 같은 분류 문제를 확률적으로 표현하는 데 사용
  - *기울기 소멸 문제* 가 발생하여 현재는 딥러닝 모델에서는 잘 사용하지 않음



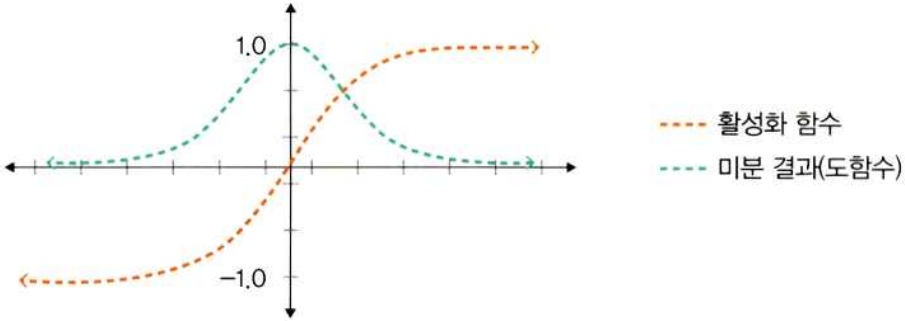
c.f. 기울기 소멸 문제(vanishing gradient problem)

- 신경망 활성화 함수의 도함수 값이 계속 곱해지다 보면 가중치에 따른 결과값의 기울기가 0에 가까워지며, 입력층으로 갈수록 기울기가 너무 작아져 가중치를 변경할 수 없게 되는 현상
- 시그모이드 함수의 도함수 값은 무조건 1보다 작기 때문에 작은 값을 계속 곱하면서 기울기가 작아지고, 기울기 소멸 문제가 발생

#4.2.1 딥러닝 용어

활성화 함수 (Activation function)

- **하이퍼볼릭 탄젠트(tanh) 함수** $f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
  - 선형 함수의 결과를 (-1)~1 사이에서 비선형 형태로 변형해줌
  - 시그모이드에서 결과값의 평균이 0이 아닌 양수로 편향된 문제를 해결
  - *기울기 소멸 문제* 는 여전히 발생



#4.2.1 딥러닝 용어

활성화 함수 (Activation function)

- **소프트맥스(Softmax) 함수** $y_k = \frac{exp(a_k)}{\sum_{i=1}^{n} exp(a_i)}$
  - 입력값을 0~1 사이에 출력되도록 정규화하여 출력 값들의 총합이 항상 1이 되도록 함
  - 보통 딥러닝에서 출력 노드의 활성화 함수로 많이 사용됨

예를 들어, 네 개의 클래스(라벨 1, 2, 3, 4)에 대해 모델이 다음과 같은 점수를 예측했다고 가정
$z = [2.0, 1.0, 0.1, 0.5]$
각 값에 소프트맥스 함수 적용

1. 모든 원소의 지수 함수 값을 계산
   $e^{2.0} = 7.389, e^{1.0} = 2.718, e^{0.1} = 1.105, e^{0.5} = 1.649$
2. 합계 계산
   $7.289 + 2.718 + 1.105 + 1.659 = 12.861$
3. 각 원소를 합으로 나눠 확률을 계산
   $\sigma(2.0) = \frac{7.389}{12.861} = 0.575$
   $\sigma(1.0) = \frac{2.718}{12.861} = 0.211$
   $\sigma(0.1) = \frac{1.105}{12.861} = 0.086$
   $\sigma(0.5) = \frac{1.649}{12.861} = 0.128$

즉, 모델이 클래스 1에 속할 확률이 57.5%, 클래스 2는 21.1%, 클래스 3은 8.6%, 클래스 4는 12.8%라고 예측

#4.2.1 딥러닝 용어

함수

- **활성화 함수 (Activation function)**
  - 전달 함수에서 신호를 입력받아 이를 적절히 처리하여 출력해 주는 함수
  - 시그모이드 함수, 하이퍼볼릭 탄젠트 함수, ReLU 함수 등이 있음

- **손실 함수 (Loss function)**
  - 학습을 통해 얻은 데이터의 추정치가 실제 데이터와 얼마나 차이가 나는지 평가하는 지표
  - 0에 가까울수록 추정이 정확함
  - 손실함수를 최소화하기 위해 *경사하강법* 사용
    e.g. 경사하강법 : 학습률(η)과 손실 함수의 미분의 기울기를 이용해 오차를 비교하고, 오차를 최소화하는 방향으로 가중치 업데이트

  - 평균 제곱 오차(MSE), 크로스 엔트로피 오차(CEE) 등이 있음

#4.2.1 딥러닝 용어

활성화 함수 (Activation function)

- **평균 제곱 오차 (Mean Squared Error, MSE)**
  - 실제값과 예측값의 차이(error)를 제곱하여 평균 낸 것
  - 차이가 클수록 MSE 값이 커짐
    (= MSE 값이 작을수록 예측력이 좋다)
  - 회귀에서 손실 함수로 주로 사용됨

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

$$\left(\begin{array}{l} \hat{y}_i\text{: 신경망의 출력(신경망이 추정한 값)} \\ y_i\text{: 정답 레이블} \\ i\text{: 데이터의 차원 개수} \end{array}\right)$$

- 크로스 엔트로피 오차 (Cross Entropy Error, CEE)
  - 분류 문제에서 원-핫 인코딩을 했을 때만 사용할 수 있는 오차 계산법
  - 정답 클래스 y_i 는 0 또는 1

$$CrossEntropy = -\sum_{i=1}^{n} y_i \log \hat{y}_i$$

$$\left(\begin{array}{l} \hat{y}_i\text{: 신경망의 출력(신경망이 추정한 값)} \\ y_i\text{: 정답 레이블} \\ i\text{: 데이터의 차원 개수} \end{array}\right)$$

#4.2.1 딥러닝 용어

활성화 함수 (Activation function)

- **크로스 엔트로피 오차 (CEE)** $CrossEntropy = -\sum_{i=1}^{n} y_i log(\hat{y}_i)$
  - A, B, C 3개의 클래스가 있는 분류 문제에서 **정답 클래스를 A** 라고 가정
  - 정답: y = [1, 0, 0]

예측이 정확할 경우

모델 예측 확률 : $\hat{y}$ = [0.7, 0.2, 0.1]

$$L = -(1 \cdot \log 0.7 + 0 \cdot \log 0.2 + 0 \cdot \log 0.1)$$

$$L = -\log 0.7$$

$$L \approx -(-0.357) = 0.357$$

- L = 0.357로 손실이 작음

예측이 잘못됐을 경우

모델 예측 확률 : $\hat{y}$ = [0.1, 0.2, 0.7]

$$L = -\log 0.1$$

$$L \approx -(-2.302) = 2.302$$

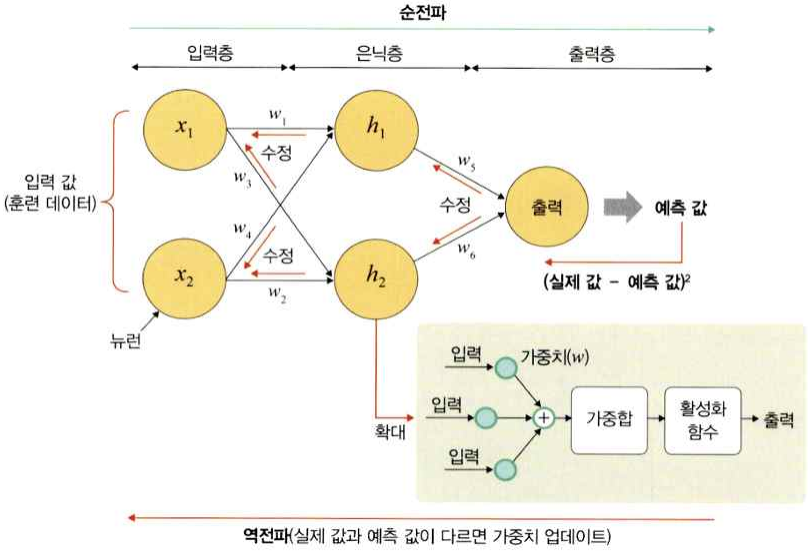
- L = 2.302로 손실이 커짐

4.2.2 딥러닝 학습

EWHA EURON

#4.2.2 딥러닝 학습

[순전파와 역전파]



순전파(feedforward): 모든 뉴런이 이전 층의 뉴런에서 수신한 정보에 변환(가중합 및 활성화 함수)을 적용해 다음 층(은닉층)의 뉴런으로 전송하는 방식

1. 네트워크를 통해 입력 데이터 전달
2. 데이터가 모든 층을 통과해 계산 완료 시 그 예측 값은 최종 층에 도달
3. **손실 함수로 네트워크의 예측 값과 실제 값의 차이(손실, 오차)를 추정**
   ⇒ 손실함수 비용을 0에 가깝도록 하기 위해 모델이 훈련 반복하며 가중치 조정

최종 정보가 역으로 전파됨 == 역전파

1. 출력층의 손실 비용 -> 은닉층 모든 뉴런으로 전파
2. 은닉 층의 뉴런은 각 뉴런이 **원래 출력에 기여한 상대적 기여도에 따라** 값이 달라짐

= (예측값-실제값)을 뉴런의 가중치로 미분-> 기존 가중치 값에서 뺌

⇒ 이 과정으로 출력층-> 은닉층-> 입력층 순서로 모든 뉴런에 진행
⇒ 각 뉴런 결과를 순전파의 가중치 값으로 사용

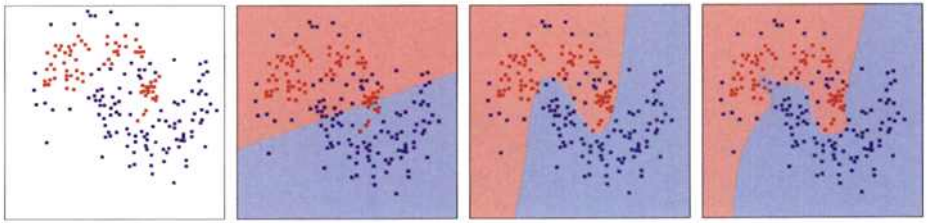
4.2.3 딥러닝의 문제점과 해결방안

EWHA EURON

#4.2.3 딥러닝의 문제점과 해결 방안

딥러닝의 핵심
: **활성화 함수가 적용된 여러 은닉층을 결합해 비선형 영역을 표현하는 것**

그림 4-13 은닉층이 분류에 미치는 영향

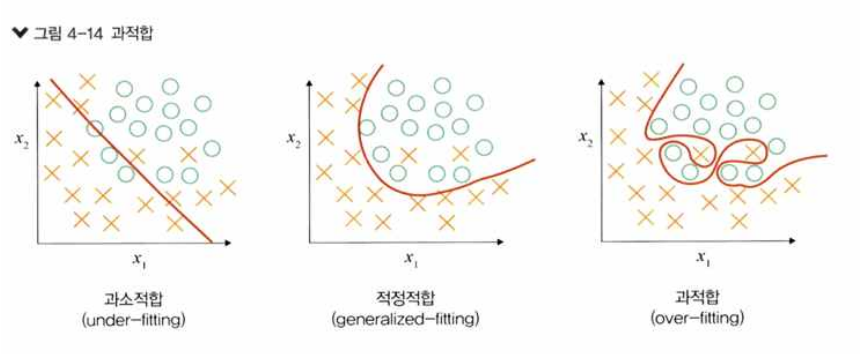


But, 은닉층이 많을 수록 3가지 문제점이 발생

1) **과적합 문제**
2) **기울기 소멸 문제**
3) **성능 저하**

#1) 과적합 문제

과적합(over-fitting) : 훈련 데이터를 과하게 학습하며 예측 값과 실제 값 차이 감소 but,
검증 데이터에 대해서는 오차가 증가

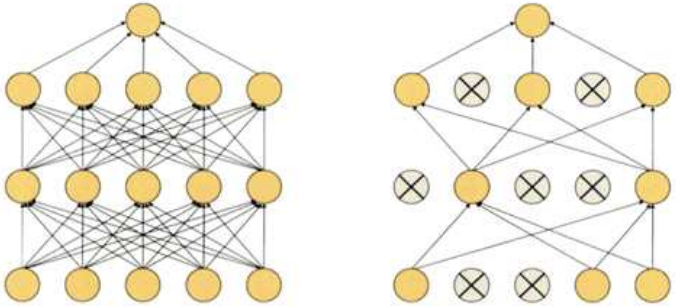


=> 해결 방법: **드롭아웃(dropout)**

#1) 과적합 문제 해결: 드롭아웃(dropout)

드롭아웃(dropout): 학습 과정 중 임의로 일부 노드들을 학습에서 제외시킴

▼ 그림 4-15 일반적인 신경망과 드롭아웃이 적용된 신경망



일반적인 신경망　　드롭아웃이 적용된 신경망

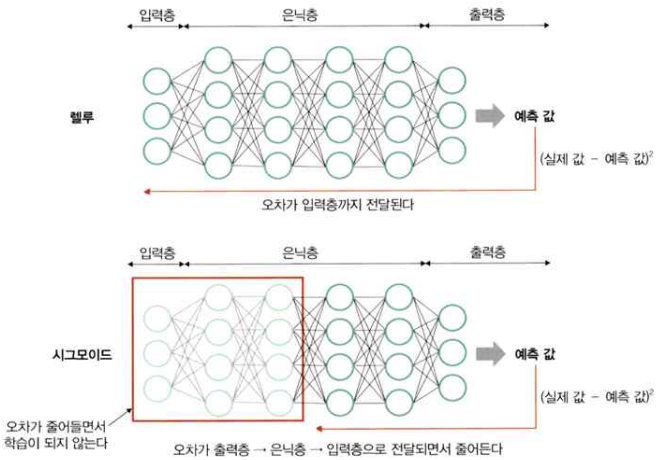
```
class DropoutModel(torch.nn.Module):
    def __init__(self):
        super(DropoutModel, self).__init__()
        self.layer1 = torch.nn.Linear(784, 1200)
        self.dropout1 = torch.nn.Dropout(0.5)
        self.layer2 = torch.nn.Linear(1200, 1200)
        self.dropout2 = torch.nn.Dropout(0.5)
        self.layer3 = torch.nn.Linear(1200, 10)

    def forward(self, x):
        x = F.relu(self.layer1(x))
        x = self.dropout1(x)
        x = F.relu(self.layer2(x))
        x = self.dropout2(x)
        return self.layer3(x)
```

50%의 노드를
무작위로 선택하여
사용하지 않겠다는 의미

#2) 기울기 소멸 문제 발생

- **<u>은닉층이 많은 신경망에서 주로 발생</u>**
- 출력층에서 은닉층으로 전달되는 오차가 크게 줄어들어 학습이 되지 않는 현상
- 기울기가 소멸되기 때문에 학습되는 양이 0에 가까워져 학습이 더디게 진행되다
  <u>오차를 더 줄이지 못하고 그 상태로 수렴</u>

❮ 그림 4-16 기울기 소멸 문제



⇒ Sigmod나 하이퍼볼릭 탄젠트 대신
ReLU 활성화 함수 사용

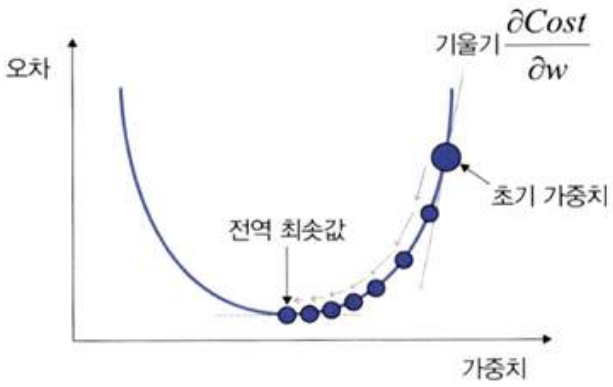
#3) 성능 저하 문제

경사 하강법

: 손실 함수의 비용이 최소가 되는 지점을 찾을 때까지 기울기가 낮은 쪽으로 계속 이동시킴

=> <u>성능 저하 문제 발생</u>

그림 4-17 경사 하강법



해결 방법

1) 확률적 경사 하강법

2) 미니 배치 경사 하강법 사용

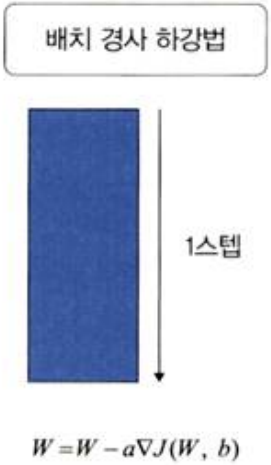
#3) 성능 저하 문제

배치 경사 하강법(Batch Gradient Descent, BGD)
: 전체 데이터셋에 대한 오류를 구한 후 기울기를 한 번만 계산해 모델의 파라미터를 업데이트하는 방법
: 전체 훈련 데이터셋에 대해 가중치를 편미분하는 방법

손실 함수의 값을 최소화하기 위해 기울기(▽) 이용

$$W = W - a\nabla J(W,\ b)$$

(a: 학습률, J: 손실 함수)



한 스텝에 모든 훈련 데이터셋을 사용하므로 학습이 오래 걸림
=>이를 보완한 방법: **<u>확률적 경사 하강법</u>**

#3) 성능 저하 문제

확률적 경사 하강법(Stochasitc Gradient Descent, SGD)
: 임의로 선택한 데이터에 대해 기울기를 계산하는 방법
: 적은 데이터를 사용하므로 빠른 계산 가능

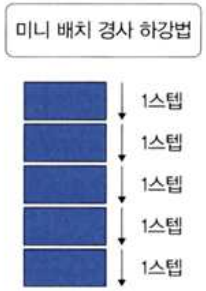
$$W = W - a\nabla J(W, b, x^{(i)}, y^{(i)})$$

그림 4-19 배치 경사 하강법과 확률적 경사 하강법

- 파라미터 변경 폭 불안정
- 배치 경사 하강법보다 정확도가 낮을 수 있음 but, 속도가 빠름

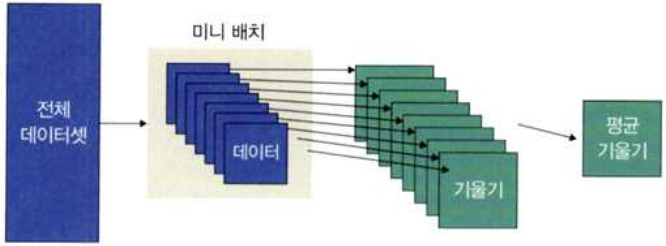
#3) 성능 저하 문제

미니 배치 경사 하강법(mini-batch gradient descent)
: 전체 데이터셋을 **미니 배치(mini-batch)** 여러 개로 나누고,
미니 배치 한 개마다 기울기를 구한 후 그것의 평균 기울기를 이용해 모델을 업데이트해서 학습



$$W = W - a\nabla J(W,\ b,\ x^{(z:z+bs)},\ y^{(z:z+bs)})$$

그림 4-20 미니 배치 경사 하강법



전체 데이터를 계산하는 것보다 빠르며, 확률적 경사 하강법보다 안정적

#3) 성능 저하 문제

파이토치에서 미니 배치 경사 하강법 구현

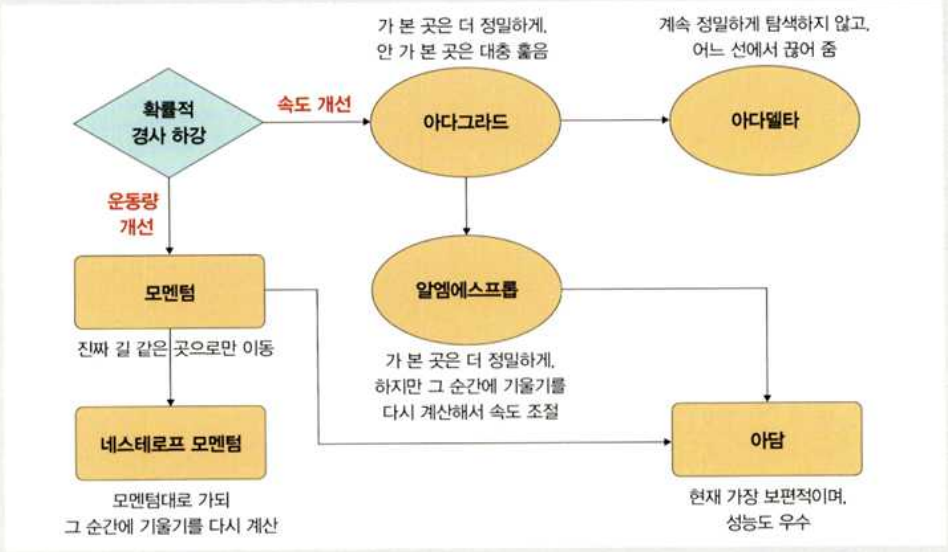
```
class CustomDataset(Dataset):
    def __init__(self):
        self.x_data = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
        self.y_data = [[12], [18], [11]]
        def __len__(self):
            return len(self.x_data)
        def __getitem__(self, idx):
            x = torch.FloatTensor(self.x_data[idx])
            y = torch.FloatTensor(self.y_data[idx])
            return x, y
dataset = CustomDataset()
dataloader = DataLoader(
    dataset, ····· 데이터셋
    batch_size=2, ····· 미니 배치 크기로 2의 제곱수를 사용하겠다는 의미입니다.
    shuffle=True, ····· 데이터를 불러올 때마다 랜덤으로 섞어서 가져옵니다.
)
```

그림 4-21 확률적 경사 하강법과 미니 배치 경사 하강법

#옵티마이저

옵티마이저
: 학습 속도와 운동량을 조정
: 확률적 경사 하강법의 파라미터 변경 폭이 불안정한 문제 해결

그림 4-22 옵티마이저 유형



#옵티마이저-속도 조절 방법

1) 아다그라드(Adagrad, Adaptive gradient)

: **변수(가중치)의 업데이트 횟수**에 따라 학습률을 조정

변수들의 변화량 ***<-반비례->*** 학습률

- 많이 변화한 변수는 최적 값에 근접했을 것이라는 가정 하에 작은 크기로 이동
- 적게 변화한 변수는 학습률을 크게 해 빠르게 오차 값을 줄임

$$w(i+1) = w(i) - \frac{\eta}{\sqrt{G(i)+\varepsilon}}\nabla E(w(i))$$

$$G(i) = G(i-1) + (\nabla E(w(i)))^2$$

G: 이전 G 값의 기울기 크기의 누적
파라미터마다 다른 학습률 주기 위함

```
optimizer = torch.optim.Adagrad(model.parameters(), lr=0.01)
```
------ 학습률 기본값은 1e-2

=> but, 기울기가 0에 수렴하는 문제 발생 **-> 알엠에스프롭** 사용

#옵티마이저 -속도 조절 방법

2) 아다델타(Adadetta, Adaptive delta)

: 아다그라드에서 G 값이 커짐에 따라 학습이 멈추는 문제를 해결하기 위해 등장한 방법

$$w(i+1) = w(i) - \frac{\sqrt{D(i-1)+\varepsilon}}{\sqrt{G(i)+\varepsilon}} \nabla E(w(i))$$

$$G(i) = \gamma G(i-1) + (1-\gamma)(\nabla E(w(i)))^2$$

$$D(i) = \gamma D(i-1) + (1-\gamma)(\Delta(w(i)))^2$$

학습률을 D 함수(가중치의 변화량의 크기를 누적한 값)로 변환했기에
학습률에 대한 하이퍼파라미터 필요X

```
optimizer = torch.optim.Adadelta(model.parameters(), lr=1.0)
```
------ 학습률 기본값은 1.0

#옵티마이저 -속도 조절 방법

3) 알엠에스프롭(RMSProp)

: 아다그라드의 G(i) 값이 무한히 커지는 것을 방지

$$w(i+1) = w(i) - \frac{\eta}{\sqrt{G(i)+\varepsilon}} \nabla E(w(i))$$

$$G(i) = \boxed{\gamma} G(i-1) + (1-\gamma)(\nabla E(w(i)))^2$$

G 함수에 감마 추가
감마 값을 이용해 학습률 크기를 비율로 조정

```
optimizer = torch.optim.RMSprop(model.parameters(), lr=0.01) ------ 학습률 기본값은 1e-2
```

#옵티마이저 -운동량 조절 방법

모멘텀(Momentum)

: 가중치를 수정하기 전에 **이전 수정방향(+,-)**을 참고해 같은 방향으로 일정한 비율만 수정하는 방법

➔ 수정이 (+) ->(-) 지그재그 현상 줄어듦

➔ 이전 이동 값을 고려해 일정 비율만큼 다음 값을 결정 -> **관성 효과**

➔ **SGD(확률적 경사 하강법)**와 함께 사용

$$w(i+1) = w(i) - \eta \nabla E(w(i))$$

가중치 계산: **기울기의 크기와 반대방향만큼** 가중치 업데이트(기울기 크면 (-) 방향으로 업데이트)

$$w(i+1) = w(i) - v(i)$$
$$v(i) = \gamma v(i-1) + \eta \nabla E(w(i))$$

SGD 모멘텀은 확률적 경사 하강법에서 기울기를 속도로 대체해 사용
이전 속도의 일정 부분 반영

```
optimizer = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
```

Momentum 값은 0.9에서 시작해 0.95, 0.99처럼 조금씩 증가

#옵티마이저 -운동량 조절 방법

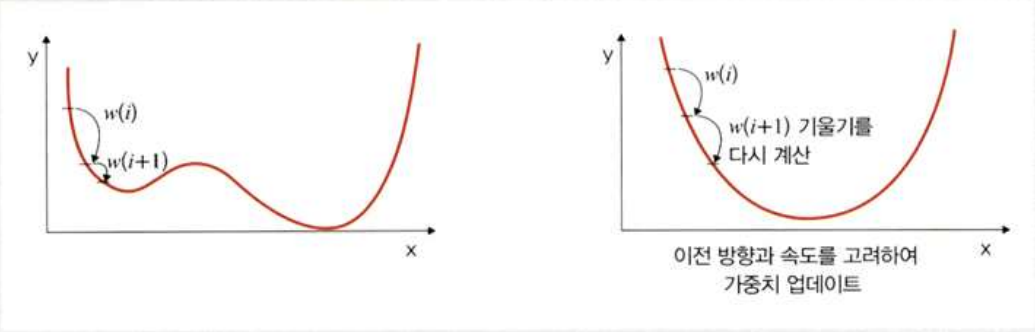
네스테로프 모멘텀(Nesterov Accelerated Gradient, NAG)
: 모멘텀 값이 적용된 지점에서 기울기 값 계산

모멘텀으로 절반 정도 이동한 후 어떤 방식으로 이동해야 하는지 다시 계산해 결정(**제동**)
➔ 멈추어야할 시점에도 관성에 의해 훨씬 멀리 갈 수 있는 단점이 있는 모멘텀 방법 극복

$$w(i+1) = w(i) - v(i)$$
$$v(i) = \gamma v(i-1) + \eta \nabla E(w(i) - \gamma v(i-1))$$

(이전 학습 속도)+(변화량)((현재 기울기)-(이전 속도)) 반영

그림 4-23 모멘텀과 네스테로프 모멘텀



#옵티마이저 -운동량 조절 방법

네스테로프 모멘텀(PyTorch)

```
optimizer = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9,
                            nesterov=True) ------ nesterov 기본값은 False
```

속도와 운동량에 대한 혼용 방법

아담(Adam, Adaptive Moment Estimation)

: 모멘텀 + 알엠에스프롭의 장점 결합한 경사 하강법

$$w(i+1) = w(i) - \frac{\eta}{\sqrt{G(i)+\varepsilon}} v(i)$$

$$G(i) = \gamma_2 G(i-1) + (1-\gamma_2)(\nabla E(w(i)))^2$$

$$v(i) = \gamma_1 v(i-1) + \eta \nabla E(w(i))$$

기울기 제곱을 지수평균 (알엠에스프롭 특징)

V(i) (모멘텀 특징)

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.01) ------ 학습률 기본값은 1e-3
```

#4.2.4 딥러닝을 사용할 때 이점

1) 특정 추출

: input 데이터를 컴퓨터가 인지할 수 있는 데이터로 변환할 때, 그 특징을 찾아내고 그것을 토대로 데이터를 벡터로 변환하는 작업

딥러닝은 ML에 있었던 이러한 **특성 추출 과정을 알고리즘에 통합**시킴
데이터 특성을 잘 잡아내고자 **은닉층을 깊게 쌓은 방식으로 파라미터 늘린 모델 구조 덕분**

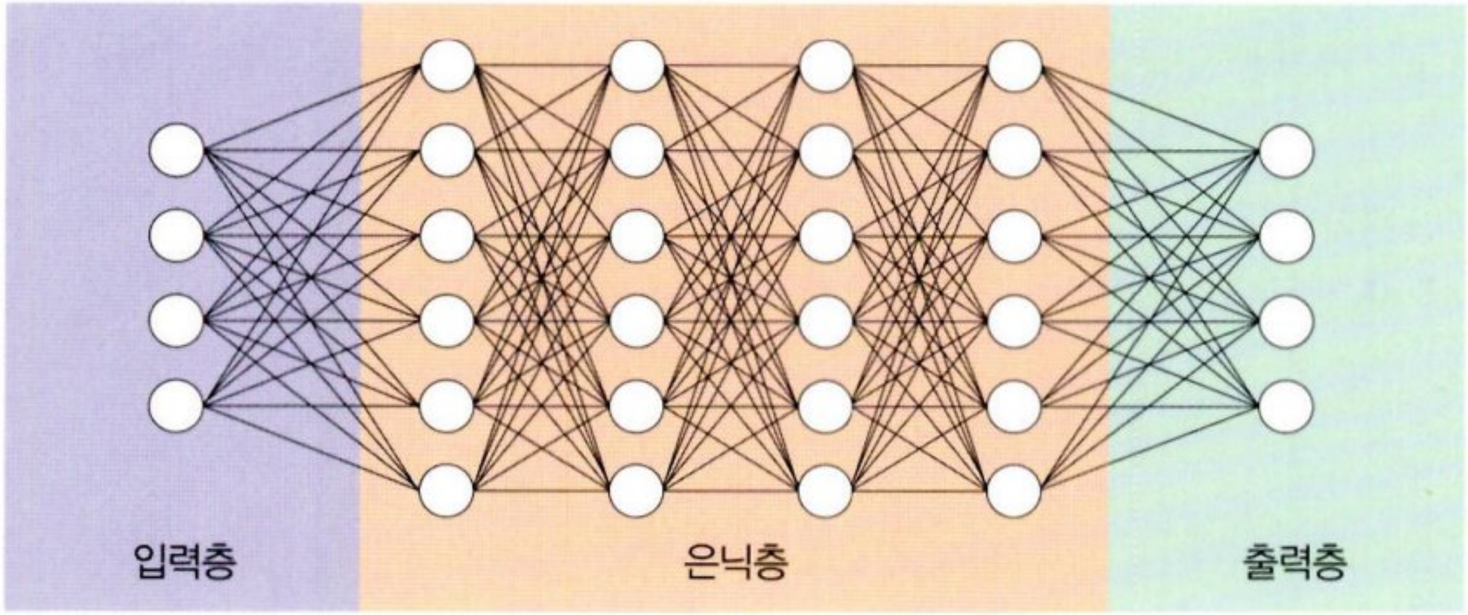
2) 빅테이터의 효율적 활용

: 빅데이터 덕분에 딥러닝에서 특성추출을 알고리즘에 통합시킴
딥러닝 학습을 이용한 특성 추출은 **데이터 사례가 많을수록 성능이 향상됨**

4.3 딥러닝 알고리즘

#4.3.1 심층 신경망 (DNN)

그림 4-24 심층 신경망



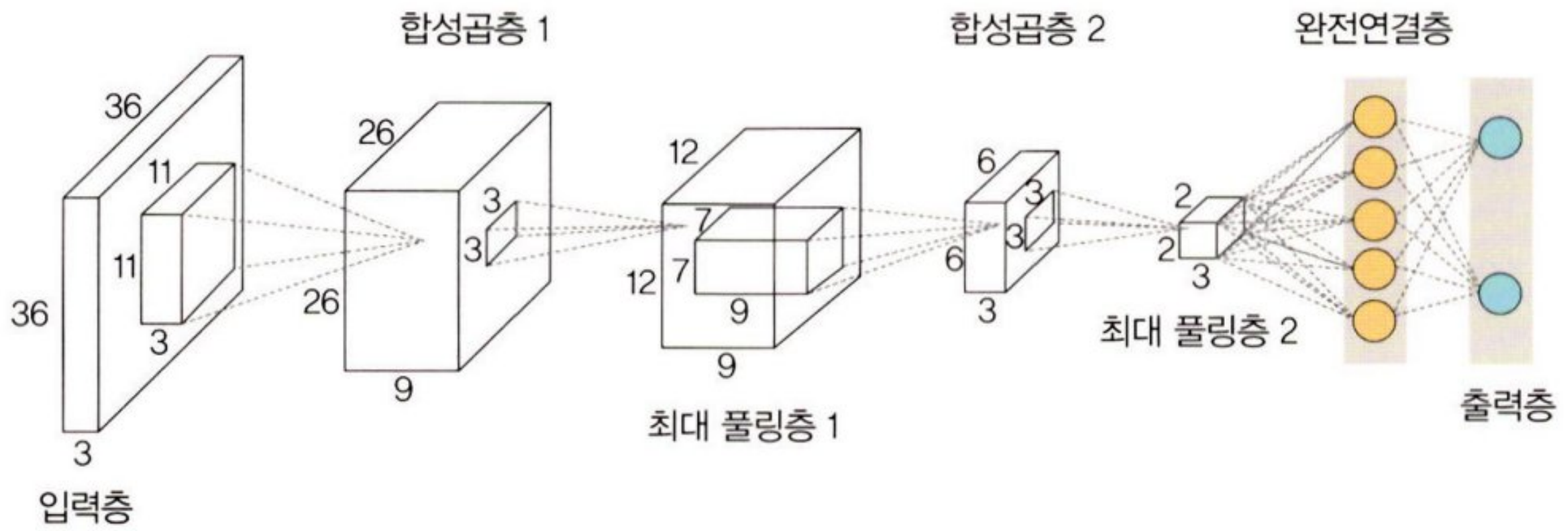
심층 신경망: 입력층과 출력층 사이에 다수의 은닉층 포함하는 인공 신경망. 다수의 은닉층 추가했기 때문에 트릭 없이 비선형 분류 가능.

단점: 학습 연상량 많음, 기울기 소멸 문제

-> 해결 방안: 드롭아웃, 렐루 함수, 배치 정규화

#4.3.2 합성곱 신경망 (CNN)

그림 4-25 합성곱 신경망



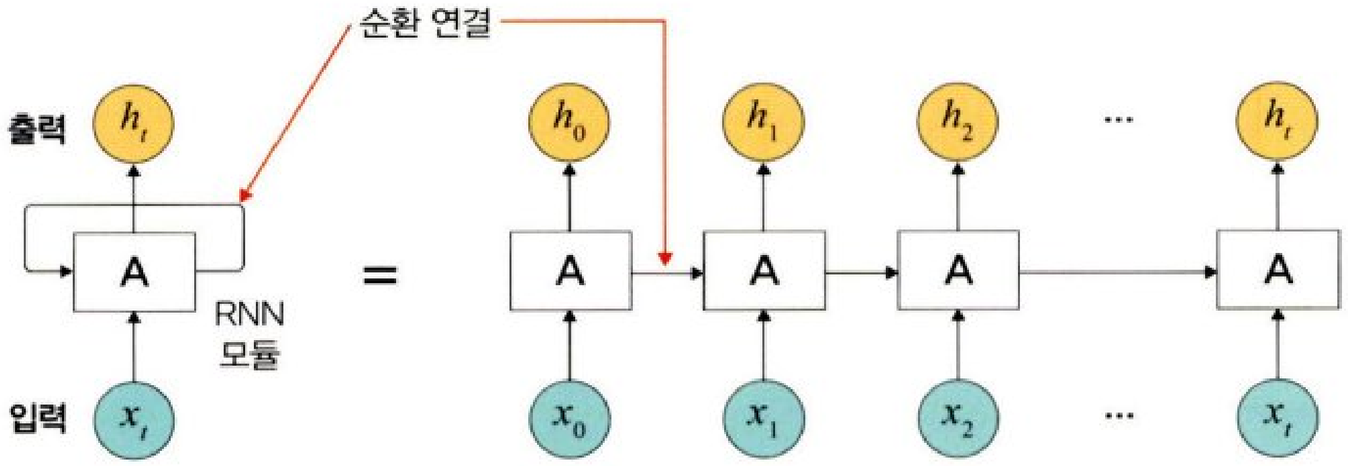
합성곱 신경망: 합성곱층 (Convolutional layer) 과 풀링층 (Pooling layer) 을 포함하는 이미지 처리 인공 신경망 알고리즘. 영상 및 사진이 포함된 이미지 데이터에서 객체 (위치) 탐색.

대표 CNN: LeNet-5, AlexNet, VGG, GoogLeNet, ResNet

기존 신경망과의 차별성: 각 층의 입출력 형상 유지, 복수 필터로 이미지 특징 추출, 필터를 공유 파라미터로 사용하므로 일반 인공 신경망보다 학습 파라미터 적음

#4.3.3 순환 신경망 (RNN)

그림 4-26 순환 신경망



- 순환(recurrent): 자기 자신을 참조,
현재 결과가 이전 결과와 연관이 있다는 의미

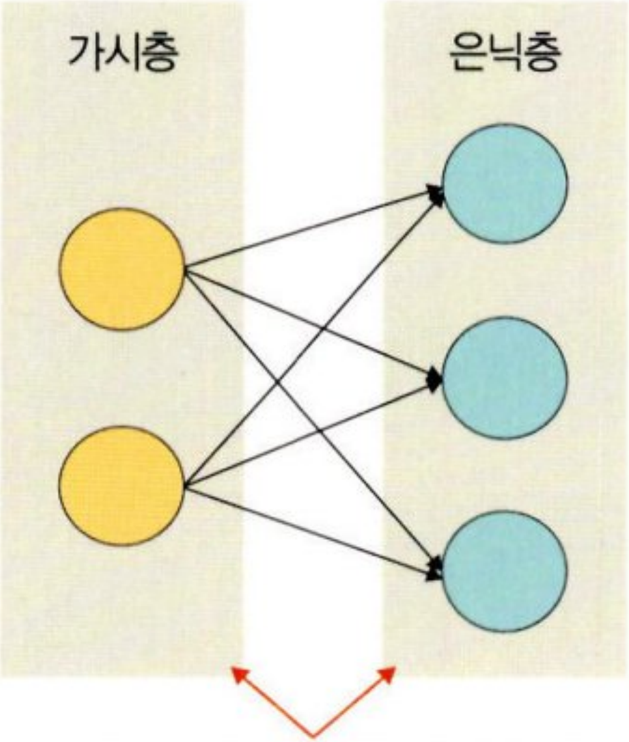
< 순환 신경망의 특징 >

1. 시간성(temporal property)을 가진 데이터가 많음
2. 시간성 정보를 이용하여 데이터의 특징을 잘 다룸
3. 시간에 따라 내용이 변하므로 데이터는 동적이고 길이가 가변적임
4. 매우 긴 데이터를 처리하는 연구가 활발히 진행되고 있음

- 기울기 소멸 문제로 학습이 제대로 되지 않는 문제가 있음
- 이를 해결하고자 메모리 개념을 도입한 LSTM이 순환 신경망에서 많이 사용되고 있음
- 자연어 처리 분야에서 많이 사용됨

#4.3.4 제한된 볼츠만 머신

그림 4-27 제한된 볼츠만 머신



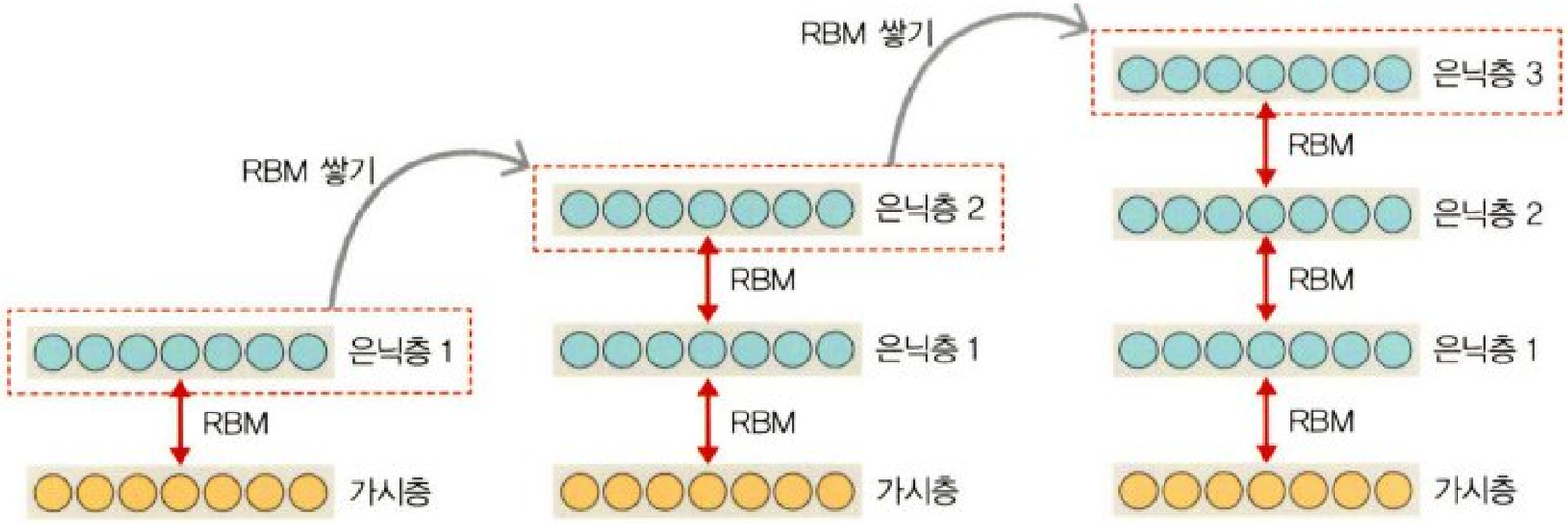
- 볼츠만 머신: 가시층(visible layer)과 은닉층(hidden layer)으로 구성된 모델
  - 가시층은 은닉층과만 연결됨
  - 가시층과 가시층, 은닉층과 은닉층 사이에 연결이 없는 모델
  = 제한된 볼츠만 머신

<제한된 볼츠만 머신의 특징>

1. 차원 축소, 분류, 선형 회귀 분석, 협업 필터링, 특성 값 학습, 주제 모델링에 사용
2. 기울기 소멸 문제를 해결하기 위해 사전 학습 용도로 활용 가능
3. 심층 신뢰 신경망(DBN)의 요소로 활용됨

#4.3.5 심층 신뢰 신경망

그림 4-28 심층 신뢰 신경망

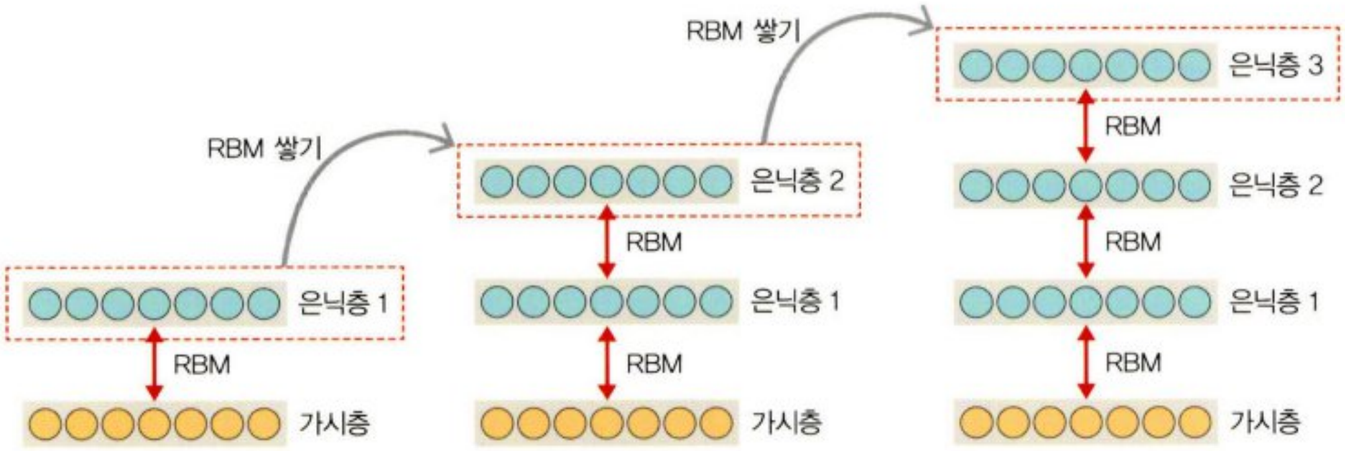


심층 신뢰 신경망: 제한된 볼츠만 머신을 블록처럼 여러 층으로 쌓은 형태로 연결된 신경망

<구조>
- 사전 훈련된 제한된 볼츠만 머신을 층층이 쌓아 올린 구조
- 레이블이 없는 데이터에 대한 비지도 학습이 가능
- 부분적인 이미지에서 전체를 연상하는 일반화와 추상화 과정을 구현할 때 유용

#4.3.5 심층 신뢰 신경망

그림 4-28 심층 신뢰 신경망



<학습 절차>

- 가시층과 은닉층 1에 제한된 볼츠만 머신을 사전 훈련 시킴
- 첫 번째 층 입력 데이터와 파라미터를 고정하여 두 번째 층 제한된 볼츠만 머신을 사전 훈련함
- 원하는 층 개수만큼 제한된 볼츠만 머신을 쌓아 올려 전체 DBN을 완성함

<심층 신뢰 신경망의 특징>

- 순차적으로 심층 신뢰 신경망을 학습시켜 가면서 계층적 구조를 생성
- 비지도 학습으로 학습
- 위로 올라갈수록 추상적 특성 추출
- 학습된 가중치를 다층 퍼셉트론의 가중치 초깃값으로 사용

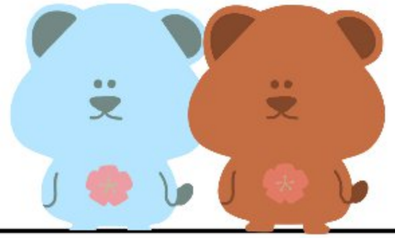
#4.3.4 우리는 무엇을 배워야 할까?

- 주어진 데이터에 따라 머신러닝을 학습할지 딥러닝을 학습할지를 결정할 수 있음
  - 간단한 선형 회귀 분류 >> 머신러닝으로 충분
  - 복잡한 비선형 데이터에 대한 분류 및 예측 >> 딥러닝으로 학습

- 머신러닝과 딥러닝은 얻고 싶은 결과를 도출하기 위한 도구일 뿐

- 데이터를 활용하여 얻고자 하는 것에 따라 머신러닝이나 딥러닝을 선택해서 학습하고 데이터를 훈련시키면 됨

THANK YOU



EWHA EURON